Computers and Electrical Engineering 130 (2026) 110838

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

Metastack-aptnet: An ensemble deep learning framework for advanced persistent threat detection and mitigation in cyber-physical systems using blockchain technology

Diana Arulkumar [a,*], Kartheeban K [b]

[a] Research Scholar, Department of Computer Science and Engineering, Kalasalingam Academy of Research and Education, Srivilliputtur, Tamilnadu, India

[b] Associate Professor, Department of Computer Science and Engineering, Kalasalingam Academy of Research and Education, Srivilliputtur, Tamilnadu, India

ARTICLE INFO

Keywords:
APT
Threat mitigation
Ensemble deep learning
Hybrid optimization
CPS
Blockchain

ABSTRACT

Advanced Persistent Threats (APTs) are increasingly targeting Cyber-Physical Systems (CPS) that drive the automation of industries, healthcare IoT, and smart grids. These are multi-stage stealthy attacks that circumvent the old systems of intrusion detection posing serious threats to the continuity and safety of operations. To solve this problem, this paper introduces MetaStack-APTNet, a new ensemble deep learning model, which is capable of not only identifying APTs with high accuracy but also scoring and mitigating them in real time. In comparison with the current models, MetaStack-APTNet is a hybrid optimization (Dynamic Addax-Flood Optimization) coupled with adaptive multi-agent decision-making and blockchain-based secure storage to provide an end-to-end defense pipeline. Experiments on real-world CPS data prove to more accurate (98.92) and resilient, with a detection time of 200 ms and mitigation time of 700 ms, which is better than the state-of-the-art baselines. Besides, SHapley Additive Explanations (SHAP) interpretability is transparent to allow practical implementation. The work contributes to the development of CPS security considerably because it offers a strong, explainable, and scalable mechanism of defense against dynamic APT campaigns and is applicable to a wide range of industrial and critical infrastructure sectors. The implementation is available at: https://github.com/diana2347a34-cell/METASTACK-APTNET-AN-ENSEMBLE-DEEP-LEARNING-FRAME-FOR-APT-DETECTION-AND-MITIGATION-USING-Blockchain.

1. Introduction

Cyber-Physical Systems (CPS) is a combination of physical processes, computational intelligence, and networking, which allows smart grids, autonomous transportation, and automation of industries [1]. Nevertheless, CPS is severely exposed to Advanced Persistent Threats (APTs), which are covert, focused, and multi-phase cyberattacks that compromise the integrity of a system [2]. The traditional machine learning models and conventional Intrusion Detection Systems (IDS) are not very effective in detecting such adaptive threats because they are based on fixed features and fixed models [3].

Machine learning techniques like Support Vector Machines, Decision Trees, and Random Forests, signature-based IDS, and

* Corresponding author.
E-mail addresses: arul.diana@gmail.com (D. Arulkumar), k.kartheeban@klu.ac.in (K. K).

https://doi.org/10.1016/j.compeleceng.2025.110838
Received 10 January 2025; Received in revised form 26 September 2025; Accepted 9 November 2025
Available online 12 December 2025
0045-7906/© 2025 Published by Elsevier Ltd.

dimensionality reduction techniques such as Principal Component Analysis (PCA) are all used as current APT detection methods [4]. These methods are effective in terms of detection accuracy of known threats, but they do not address the multi-stage and adaptive nature of APTs [5–8]. Deep learning, reinforcement learning, and transfer learning-based advanced techniques have performed better in anomaly detection. Nevertheless, there are still issues such as the imbalance of data, high computational expenses, low interpretability, and the inability to keep up with the changes in attacks [9,10].

In order to overcome these challenges, this study introduces MetaStack-APTNet, a new hybrid optimization ensemble deep learning framework with threat scoring based on fuzzy logic, Adaptive Mitigation using Multi-Agent Markov Decision Processes (AM-MDP), and a secure blockchain-based data storage. The model is designed to offer a dynamic, explainable, and scalable solution to real-time detection, scoring, and mitigation of APTs in CPS environments to address the existing issues of adaptability, interpretability, and robust protection. The key contributions are:

- Introduced a multi-level detection system that integrates a hybrid meta learner (XGBoost + LR) with a stacked ensemble (CNN-LSTM-BNN) to be able to detect robustly APT.
- Added the Dynamic Addax-Flood Optimization (DAFO) to the efficient hyperparameter tuning.
- Developed an AM-MDP to real-time threat mitigation.
- On real CPS datasets (including SCADA traffic), proven and improved the credibility and feasibility of deployments.

This article is structured as a recent literature on APT detection in Section 2. Implemented framework and the description are given in Section 3. Results and the significance are presented in Section 4. Section 5 ends the research.

2. Literature study

Several approaches have been proposed in the literature for detecting and mitigating APTs, particularly using Machine Learning (ML), Deep Learning (DL), and integrated system strategies.

2.1. ML and DL-based detection approaches

In 2024 Arefin et al. [11] addressed the issue of APT detection accuracy using a variety of machine learning algorithms by testing their effectiveness in sophisticated cyber-attack identification. In 2022 Neuschmied et al. [12] used a multi-stage autoencoder technique to propose an approach toward detecting APT attacks using unsupervised learning, identifying strange patterns in network traffic.

2.2. Integrated detection and mitigation strategies

In 2023 Fahad et al. [13] pointed out the advancement of APT detection and mitigation through a complete approach, integrating multiple strategies for detection. In 2023 Lee et al. [14] discussed various detection techniques against APTs. In this work, they

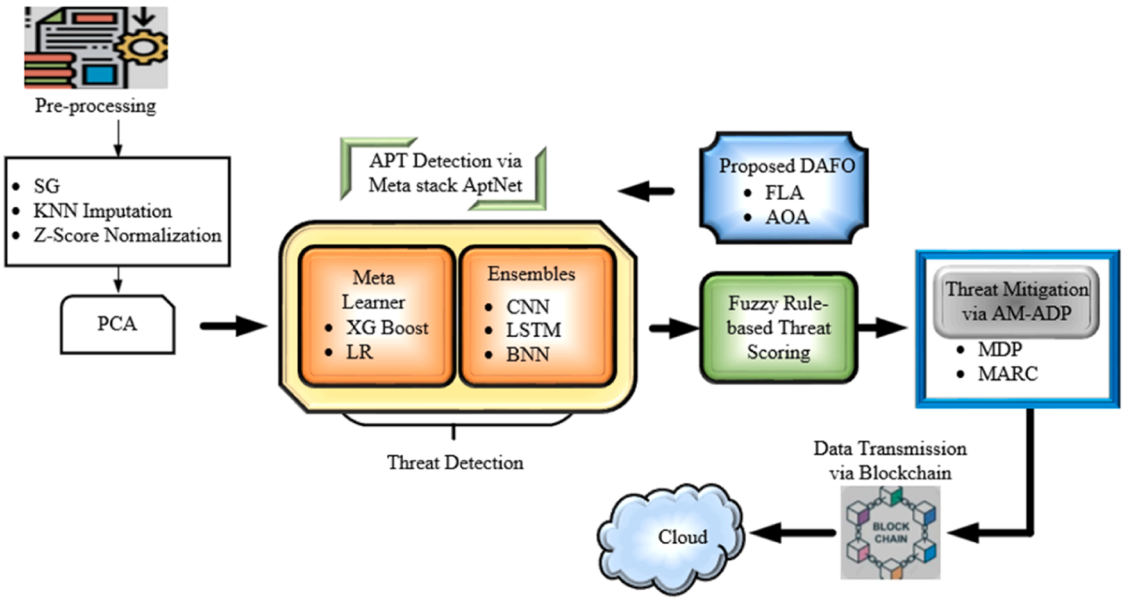


Fig. 1. Block diagram of proposed model.

categorized and analyzed various detection techniques.

2.3. Security in CPS and smart systems

In 2022 Zhang et al. [15] reviewed the progress made on industrial CPSs by indicating the key advances and challenges in securing the systems from APTs. In 2022 Lee and Kundu [16] proposed an integrated cyber-physical system that combines with the industrial metaverse for remote manufacturing.

2.4. Problem statement

APT are a serious threat to CPS because they are inconspicuous and dynamic. The current methods of detection generally require large labelled datasets, highly computationally costly, and do not adapt to new attack strategies. Majority of the frameworks do not have built-in mitigation and secure data management and can only be effective in real-world settings. Therefore, there is a necessity of a strong, readable solution that will provide correct identification of APTs and fast elimination of them within time-sensitive CPS settings.

3. A novel APT detection AND mitigation model

Fig. 1 addresses the overview of the proposed model. The proposed research presents a new APT detection model of CPS based on ensemble deep learning and advanced threat mitigation measures. Savitzky-Golay filters, KNN imputation and Z-score normalization are used to pre-process network and PLC data, then dimensionality reduction is done by PCA. MetaStack-APTNet model which is a hybrid of a meta-learner (XGBoost + LR) and stacked ensemble (CNN, LSTM, BNN) is a high-accuracy threat detector, enhanced by hyperparameter optimization with DAFO. The data on resolved threats is safely stored on the cloud through blockchain to guarantee integrity that cannot tampered.

3.1. Preprocessing

Normally, preprocessing in APT detection plays a critical role in cleaning and transforming raw network data. It ensures the removal of noise and handling of missing values.

3.1.1. SG filter

The SG smoothing filter [17] is commonly employed for eliminating and flattening information while maintaining significant features such as edges, as well as peaks. SG filtering utilizes least squares to assign consecutive subsets of neighboring data points to a polynomial, as shown in Eq. (1).

$$p(r_i) = c_0 + c_1^r + \ldots + c_p^{r^p} \tag{1}$$

Here, the associated visible resistance information vector is denoted by ρ in this composition, and the northern geographical position of the resistance map is represented by "r_i". The goal of fitting a polynomial of degree n in a sliding window with a window size $2m+1$ (odd number) and centered at i is the objective. The position in comparison to an object within the frame is denoted by x, so that $x = -m, \ldots\ldots, m$. Eq. (2) is utilized for estimating the polynomial coefficients.

$$P(x) = c_0 + c_1 x + c_2 x^2 + \ldots + c_n x^n \tag{2}$$

Eq. (3) provides the coefficients $\{c_k\}$ of the polynomials that have been identified through minimization. Here m represents the polynomial degree $m \in \mathbb{N}, m \geq 1$.

$$\sum_{j=-m}^{m} \left(y_{i+j} - P(j)\right)^2 \tag{3}$$

Eq. (4) presents the smoothed value y_i^{SG}, which is the numerical value of the polynomial at $x = 0$.

$$y_i^{SG} = P(0) = c_o \tag{4}$$

A convolution is utilized to represent the smoothing process, where Eq. (5) provides the filtering coefficients (weights) obtained from the polynomial fit h_j. Here, i is the index of the data point $i \in \{1, 2, \ldots, n\}$.

$$y_i^{SG} = \sum_{j=-m}^{m} h_j y_i + j \tag{5}$$

3.1.2. kNN imputation

In general, kNN Imputation [18] is a technique that employs points of data comparison to fill in missing data. Employing the values of the dataset's k closest information points or neighbors, this non-parametric imputation method predicts the missing values

presented in Eq. (6).

$$D_{ij} = \left[\sum_{n=1}^{N} w_n \left(x_{ni} - x_{nj}\right)^2\right]^{0.5} \tag{6}$$

Where $k_{non-missing}$ represents a collection of indices, nor x_{ni} nor x_{nj} is missing and x_{ni} nor x_{nj} denotes the value of characteristics k for samples i *and* j respectively. Here, $i,\ j$ are indices over samples and features $i \in [1, N], j \in [1, D]$ and N is the set of k-nearest neighbors of i.

3.1.3. Z-score normalization

The technique of adjusting every statistic in a collection of data so that the deviation of each value is one and the mean of all the values is zero is known as Z-score normalization [19]. Each value in an information set is normalized utilizing the z-score by Eq. (8), where x represents the original value, μ indicates the data mean, and σ represents the data standard deviation (SD).

$$New\ value = (X - \mu)/\sigma \tag{8}$$

The actual execution of z-score normalization on an information set is demonstrated by Eq. (8). The SD of the normalized numbers is one, and the mean is zero. The quantity of SD that the initial number departs from the mean is indicated by the normalized numbers.

3.1.4. PCA

PCA is a statistical method that generally decreases the dimension of information by reflecting it onto a collection of orthogonal axes (principal components) that maximizes variability [20]. It preserves the data's most important patterns while decreasing the number of attributes. The information is scaled as well as centered so that each characteristic has a unitary variance and a zero mean, provided in Eq. (9), where $x_{i,j}$ indicates the ith reflection of characteristics j, μ_j indicates the mean $\mu \in \mathbb{R}$, and σ_j represents the SD $\sigma \in \mathbb{R}^+, \sigma > 0$.

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{9}$$

Compute the covariance matrix C of the standard information in order to understand feature connections, where Z is the normalized information matrix, which has a form $n \times p$ (n samples, p features) are expressed in Eq. (10).

$$C = \frac{1}{n-1} Z^{\dagger} Z \tag{10}$$

Eq. (11) performs eigen decomposition to find eigenvalues (λ) and eigenvectors (v) of the Covariance matrix (C), where λ represents the variance explained by each principal component and v defines the directions of the principal components and V denotes eigen vector.

$$C_v = \lambda_V \tag{11}$$

The extension of the matrix W is formed by sorting the eigenvalues in decreasing order and presenting the top k eigenvectors (corresponding to the biggest eigenvalues) in Eq. (12).

$$W = [V_1,\ V_2, \ldots, V_k] \tag{12}$$

Eq. (13) represents the transformation of the initial information onto the new k-dimensional region.

$$Z_{PCA} = ZW \tag{13}$$

3.2. APT detection via proposed metastack-aptnet

The proposed MetaStack-APTNet model uses a hybrid approach that combines a meta-learner and a stacked ensemble model to detect APTs with high accuracy.

3.2.1. Hybrid meta-learner: XGBoost with LR

Traditionally, XGBoost [21] is a scalable, tree-based algorithm that optimizes decision trees using gradient boosting. Its objective function consists of a loss function $\mathscr{L} \in \mathbb{R}$ and a regularization term $\Omega \in \mathbb{R}$ as shown in Eq. (14), in which $l\left(y_i, y_i'\right)$ states the logistic loss function, and $\varphi\left(f_k\right)$ indicates a regularization term to avoid overfitting as explained in Eq. (15), where T refers to the leave count $T \in N$ and ω expresses leaf weight.

$$L(\varnothing) = \sum_{i=1}^{n} l\left(y_i, y_i'\right) + \sum_{k=1}^{K} \varphi(f_k) \tag{14}$$

$$\varphi(f_k) = \gamma T + \frac{1}{2}\lambda \| \omega \|^2 \tag{15}$$

Finally, the predictions from XGBoost are denoted as $P_{XGB}(x)$. Fig. 2 displays the implemented architecture of XGBoost.

On the other hand, LR [22] provides probabilistic predictions for binary classification. The model estimates the probability P of a class label y as shown in Eq. (16), where x refers to the feature vector $x \in \mathbb{R}^n$, w states weight $w \in \mathbb{R}^n$, and b addresses bias $b \in \mathbb{R}$.

$$P_{LR}(x) = \frac{1}{1 + e^{-(w^T x + b)}} \tag{16}$$

3.2.2. Stacked ensemble model: CNN, LSTM, and BNN

The stacked ensemble combines three DL models, such as CNN [23], LSTM [24], and BNN [25]. Here, CNN is utilized to extract spatial features from the data using convolutional layers as stated in Eq. (17), where W and B indicate filter kernel $W \in \mathbb{R}^{k \times k}$, and bias, $*$ denotes convolution and x signifies input feature map $x \in \mathbb{R}^{h \times \omega}$.

$$f_{cnn}(x) = ReLU(W * x + B) \tag{17}$$

Once the features $f_{cnn}(x)$ are extracted, they are passed through FC layers to produce the final predictions $P_{cnn}(x)$ as illustrated in Eq. (18), in which W_{fc} and B_{fc} state weights and bias of the FC layer, and *Softmax* is used to convert the output into probabilities.

$$P_{cnn}(x) = Softmax\left(W_{fc} f_{cnn}(x) + B_{fc}\right) \tag{18}$$

Fig. 3 shows the suggested CNN. As shown in Fig. 3, the CNN model is configured with 16 filters, a kernel size of 3 × 3, a learning rate of 0.001, and a dropout rate of 0.3, optimized using the proposed DAFO algorithm.

Normally, the LSTM is a type of Recurrent Neural Network (RNN) designed to handle long-term dependencies in sequential data. It maintains a hidden state h_t at each time step to capture the information relevant to predicting the next time step or making a decision. The hidden state $h_t \in \mathbb{R}^d$ is computed using input ι_t, forgot F_t and output gates $\mathbb{O}_t$ with cell $\mathbb{C}_t \in \mathbb{R}^d$ and hidden state h_t as shown in Eq. (19) to (23), in which σ indicates activation and $\odot$ denotes element-wise multiplication, and $\chi_T \in \mathbb{R}^n$ is the input at time T.

$$\iota_t = \sigma(\omega_i \cdot [h_{t-1}], \chi_T + \beta_i) \tag{19}$$

$$F_t = \sigma(\omega_f \cdot [h_t - 1], \chi_T + \beta_f) \tag{20}$$

$$\mathbb{O}_t = \sigma(\omega_o \cdot [h_t - 1], \chi_T + \beta_o) \tag{21}$$

$$\mathbb{C}_t = F_t \odot \mathbb{C}_{t-1} + \iota_t \odot \tanh(\omega_m \cdot [h_t - 1], \chi_T + \beta_m) \tag{22}$$

$$h_t = \mathbb{O}_t \tanh \odot (\mathbb{C}_t) \tag{23}$$

The final prediction $P_{LSTM}(x)$ is typically derived from the last hidden state h_T at the final time step T as expressed in Eq. (24), where W_{LSTM} and B_{LSTM} refer to weights and biases for the final prediction layer.

$$P_{LSTM}(x) = Softmax(W_{LSTM} h_T + B_{\mathrm{LSTM}}) \tag{24}$$

The proposed MetaStack-APTNet uses BNNs to add uncertainty modeling to neural networks (NNs) by introducing probabilistic weights as stated in Eq. (25), where $P(w|D)$ addresses posterior distribution $P \in [0, 1]$ of weights given the data D.

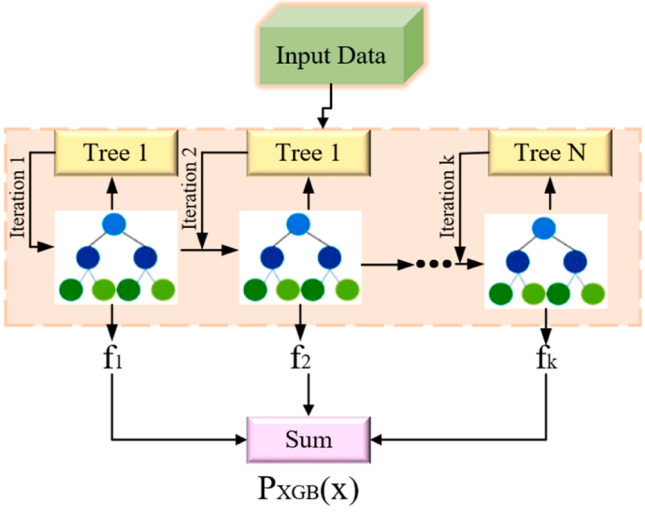


Fig. 2. Implemented architecture of XGBoost.

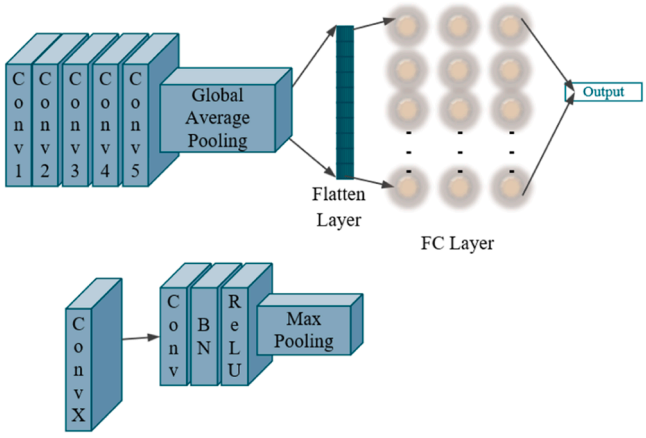


Fig. 3. Implemented CNN architecture.

$$P(y|x, W) = \int P(y|x, w)P(w|D)dw \tag{25}$$

At this point, the outputs of the base learners are combined through a weighted linear combination as expressed in Eq. (26), where $\mathscr{A}_1$, $\mathscr{A}_2$, and $\mathscr{A}_3$ points to weight coefficients optimized during training.

$$P_{stacked}(x) = \mathscr{A}_1 P_{cnn}(x) + \mathscr{A}_2 P_{LSTM}(x) + \mathscr{A}_3 P_{BNN}(x) \tag{26}$$

The hybrid meta-learner $P_{Meta}(x)$ combines the outputs of the meta-learner and stacked ensemble as formulated in Eq. (27), where $\mathscr{B}_1$, $\mathscr{B}_2$, and $\mathscr{B}_3$ refer to the weight coefficients for the hybrid integration. The weighted fusion is defined in Eq. (27). Eq. (28) gives the final classification decision, where $\theta \in [0, 1]$signifies decision threshold optimized for maximum performance.

$$P_{Meta}(x) = \mathscr{B}_1 P_{XGB}(x) + \mathscr{B}_2 P_{LR}(x) + \mathscr{B}_3 P_{stacked}(x) \tag{27}$$

$$Y = \begin{cases} 1 & if\ P_{Meta}(x) \geq \theta \\ 0 & o.w \end{cases} \tag{28}$$

The decision about APT detection Y is processed by a fuzzy rule to attain the threat score. Fig. 4 demonstrates the architecture of proposed MetaStack-APTNet. Table 1 summarizes the key symbols and parameters referenced throughout the proposed model's mathematical equations

3.3. DAFO for tuning hyperparameters

The AOA population comprising addaxes and their associated vectors is modeled as a matrix using Eq. (29), where X denotes the population matrix of AOA, $X_i \in \mathbb{R}^d$ refers to the ith addax (candidate solution), and $x_{i,d}$ signifies its dth dimension in the search space $d \in \mathbb{N}$ (decision variable), $N \in \mathbb{N}$ represents the number of addaxes in the population, m signifies the count of decision variables.

$$\mathrm{X} = \begin{bmatrix} X_1 \\ \vdots \\ X_i \\ \vdots \\ X_N \end{bmatrix} = \begin{bmatrix} x_{1,1} & \cdots & x_{1,d} & \cdots & x_{1,m} \\ \vdots & \ddots & \vdots & \iddots & \vdots \\ x_{i,1} & \cdots & x_{i,d} & \cdots & x_{1,m} \\ \vdots & \iddots & \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,d} & \cdots & x_{N,m} \end{bmatrix}_{N \times m} \tag{29}$$

Eq. (30) utilizes a random initialization of the addaxes in the distance of the solving problem, where $r \in [0, 1]$ denotes an arbitrary number in the interval [0,1], and lb_d and ub_d represent the upper and lower bounds of the dth selected factor, accordingly.

$$x_{i,d} = lb_d + r.(ub_d - lb_d) \tag{30}$$

Eq. (31) employs every addax's place as a potential approach and evaluates the parameters for the desired function by evaluating the appropriate goal function for each addax. F represents the vector of evaluated impartial operations, and F_i indicates the ranking of the goal that operates based on the ith addax.

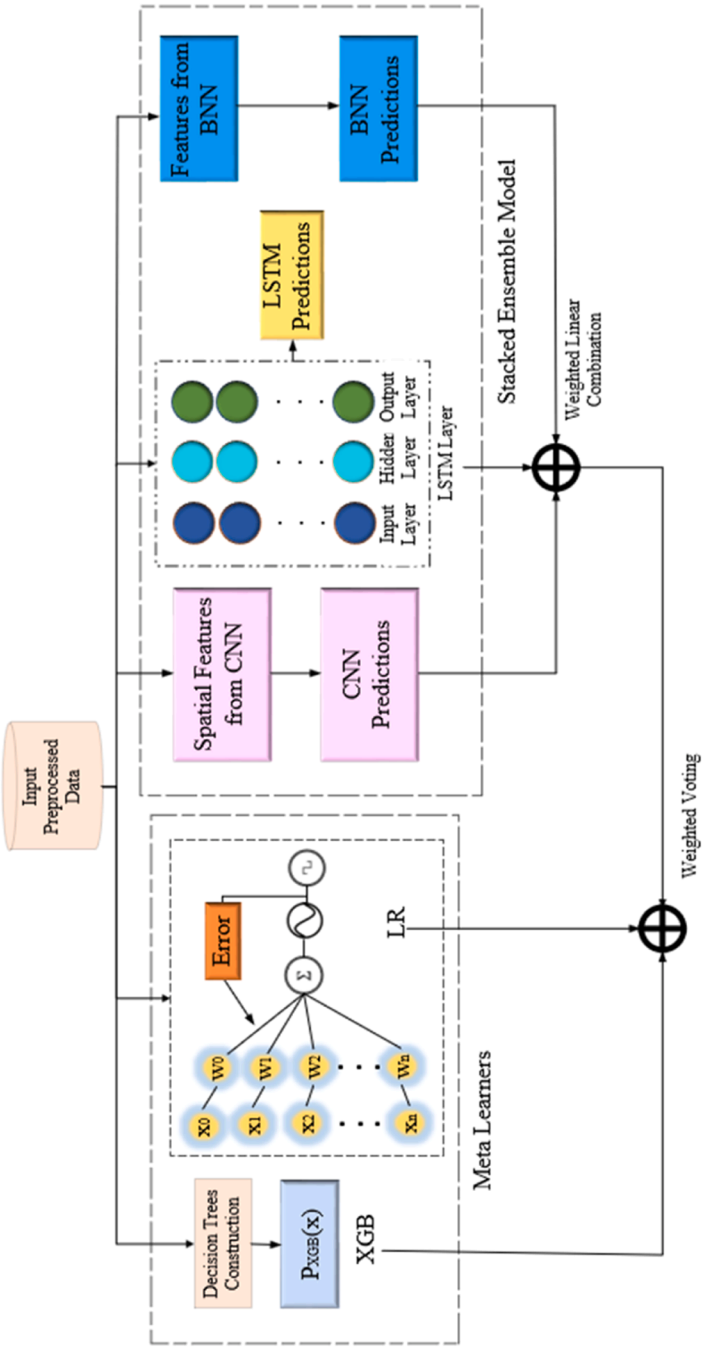


Fig. 4. Architecture of proposed MetaStack-APTNet.

Table 1
Explanation of symbols and parameters used in the mathematical formulations.

Symbol	Description
ρ	Resistance information vector used in SG filter equations
i	Index of the data point in the window
m	Polynomial degree for SG filter
w	Window size (odd number) for SG filter
x	Position of object within the data frame
x_i	Data point at position i
$f(x)$	Polynomial function used in SG filter
k	Number of nearest neighbors in k-NN imputation
z	Z-score normalized value
μ	Mean of the dataset
σ	Standard deviation of the dataset
λ	Eigenvalues in PCA representing variance along principal components
v	Eigenvectors in PCA
X	Input feature matrix
$W,\ b$	Weights and biases in DL models (CNN, LSTM, BNN)
f, σ	Activation functions (e.g., ReLU, Sigmoid)
a	Action in MDP/MARL framework
s	State in MDP/MARL framework
R	Reward function
γ	Discount factor in MDP
π	Policy function
θ	Model parameters or coefficients
η	Learning rate in optimization
α, β	Weights for combining outputs or rewards
D	Dataset used for Bayesian inference or posterior computation
$\mu_c(x)$	Membership function in fuzzy logic
COA	Centroid of Area, used in defuzzification
$H(D)$	Hash value of data

$$F = \begin{bmatrix} F_1 \\ \vdots \\ F_i \\ \vdots \\ F_N \end{bmatrix}_{N\times\ 1} = \begin{bmatrix} F(\mathrm{X}_1) \\ \vdots \\ F(\mathrm{X}_i) \\ \vdots \\ F(\mathrm{X}_N) \end{bmatrix}_{N\times\ 1} \tag{31}$$

According to Eq. (32), every addax in the AOA architecture takes into consideration the positions of the remaining addaxes with higher goal function values to determine appropriate foraging regions. The collection of possible regions with appropriate vegetation for foraging for the ith addax is represented by CA_i. The population individual having a higher goal function level than the ith addax is indicated by X_k, and its objective function value is represented by F_k, where $i = 1, 2, \ldots,\ N$ *and* $k \in \{1, 2, \ldots, N\}$.

$$CA_i = \{\mathrm{X}_k : F_k < F_i \ and\ K \neq i\} \tag{32}$$

Eq. (33) has been employed to determine the novel positions for every part of the AOA population, considering that every addax selects at random one of these factors specified regions to forage. The ith Addax's selected foraging space is indicated by SA_i, and its jth size in that space is represented by j, where x_i^{p2} indicates its jth dimension, x_i^{p1} represents the recently calculated location for i^{th} addax, which is obtained from the foraging stage of the proposed AOA. According to Eq. (34), F_i^{p1} corresponds to the goal function values for the *ith* addax in this stage. Furthermore, arbitrary numbers obtained from the range [0, 1] are represented by $r_{i,j}$, while selected at random numbers with values of 1 *or* 2 are indicated by $I_{i,j}$.

$$x_{i,j}^{p1} = x_{i,j} + r_{i,j}.\left(SA_{i,j} - I_{i,j}.x_{i,j}\right) \tag{33}$$

$$x_i = \begin{cases} x_i^{p1} \\ x_i \end{cases}, F_i^{p1} \ \leq F_i \tag{34}$$

Applying Eq. (35), the flooding and removal process is employed, in which flooding provides random disruptions that increase variety in populations while removal rejects or displaces lower desirable solutions.

$$x_i^{p2} = \mathrm{X}_i(t) + \gamma.Flooting(t) + \delta.\ Removal\ (t) \tag{35}$$

In Eq. (36), the location of $\mathrm{X}_i(t)$ is adjusted in a random orientation. *Rand* represents a random vector created from an even distribution, usually among, $[-1, 1]$ or a typical distribution. Here, ζ denotes a scaling factor that regulates the scale of the flooding (similar to the intensity of flood waters). The scope of search variety or the separation between a population's worst and greatest options is represented as $||\mathrm{X}_{max} - \mathrm{X}_{min}||$ and is employed for normalization.

$$Flooding(t) = \zeta.\left(\frac{Rand}{||X_{max} - X_{min}||}\right) \tag{36}$$

The method of removing less powerful or less-fitting substances is modelled by the removal term in Eq. (37), where τ represents a factor that regulates the force of the elimination (how aggressively less powerful ideas are eliminated), $X_i(t)$ represents the present state of the *i th* Solution $X_{worst}(t)$ At time (t), the population's lowest selection is (t).

$$Removal(t) = -\tau.(X_i(t) - X_{worst}(t)) \tag{37}$$

Eq. (38) combines the Flooding as well as eliminating phrases to obtain the updated solution, where $(\gamma.flooding\ (t))$ shows to investigate and variation through incorporating arbitrary motion, modeling the flooding operation, and $(\delta.removal(t))$ indicates to remove less powerful solutions by dragging them toward the most harmful solution.

$$X_i^{p2} = X_i(t) + \gamma.\zeta.\left(\frac{Rand}{||X_{max} - X_{min}||}\right) - \delta.\ \tau.(X_i(t) - X_{worst}(t)) \tag{38}$$

According to Eq. (39), this new position replaces the relevant member's prior place if it leads to a reduction in the desired function value. Algorithm 1 illustrates the pseudocode of the developed DAFO.

$$X_i = \begin{cases} X_i^{p2} \\ X_i \end{cases},\ F_i^{p2} \le F_i \tag{39}$$

Table 2 shows the Hyperparameter Tuning for MetaStack-APTNet. The proposed DAFO algorithm was used to obtain these values, and it is an effective combination of AOA and FLA in terms of balancing between exploration and exploitation.

3.4. Fuzzy rule-based threat scoring

The fuzzy rule-based threat scoring system is implemented to attain the severity of the threats by using a set of if-then fuzzy rules to assign a threat score Y based on input variables derived from the APT detection system. These variables are typically represented as fuzzy sets. Let the inputs to the fuzzy system $I_1, I_2, \ldots, I_n$ be indicators of potential APT activity (unusual traffic patterns, failed login attempts, file integrity violations). Each input I_i is fuzzified into membership values corresponding to linguistic variables like Low (L), Medium (M), and High (H) as shown in Eq. (40), where μ_L, μ_M, μ_H are membership functions $\mu \in [0,1]$ representing the degree to whichI_i belongs to each fuzzy set.

$$\mu_L(I_i), \mu_M(I_i), \mu_H(I_i) \tag{40}$$

Eq. (41) illustrates the fuzzy rules are construction. Each rule assigns a consequent fuzzy set for the output variable Y, which corresponds to the threat score.

$$IF\ I_1\ is\ L\ AND\ I_2\ is\ H\ THEN\ Y\ is\ M \tag{41}$$

Using fuzzy operators like AND (min) and OR (max), the fuzzy rule base evaluates the degree of activation of each rule. Eq. (42) explains a specific rule R_k.

$$R_k : IF\ I_1\ is\ F_1\ AND\ I_2\ is\ F_2\ THEN\ Y\ is\ G_k \tag{42}$$

Table 2
Hyperparameter tuning for MetaStack-APTNet.

Component	Hyperparameter	Best Solution
XGBoost	Learning Rate (η)	0.2
	Max Depth (d)	7
	Number of Trees ($n_{estimators}$)	100
LR	Regularization Strength (C)	0.7
CNN	Number of Filters	16
	Kernel Size	3×3
	Learning Rate	0.001
	Dropout Rate	0.3
LSTM	Number of Units	50
	Dropout Rate	0.4
	Learning Rate	0.01
BNN	Prior Variance	0.1
	Posterior Variance	[0.2, 0.2, 0.2]
DAFO	Population Size (N)	50
	Max Iterations	50
	Flooding Factor (γ)	0.2
	Removal Factor (τ)	0.2
	Scaling Factor (ζ)	0.3
	Random Noise Factor ($Rand$)	$[-1, 1]$

The activation level α_k of the rule R_k is formulated in Eq. (43), in which $F_1,\ F_2$ signifies the fuzzy sets for inputs $I_1,\ I_2$, and G_k addresses the output fuzzy set for Y.

$$\alpha_k = \min\left(\mu_{F_1}(I_1), \mu_{F_2}(I_2)\right) \tag{43}$$

The outputs of all fuzzy rules are aggregated to form a combined fuzzy set for Y as stated in Eq. (44), where $\mu_{G_k}(y)$ explains the membership function of the consequent fuzzy set G_k, and α_k scales the membership function based on the activation of R_k.

$$\mu_Y(y) = \max_k\left(\alpha_k \cdot \mu_{G_k}(y)\right) \tag{44}$$

The aggregated fuzzy set $\mu_Y(y)$ is defuzzified to produce a crisp threat score Y using Centroid of Area (CoA) as signified in Eq. (45). This gives the final threat score as a weighted average of all possible output values y, weighted by their membership degrees $\mu_Y(y)$.

$$Y = \frac{\int y \cdot \mu_Y(y) dy}{\int \mu_Y(y) dy} \tag{45}$$

3.5. Threat mitigation via proposed AM-MDP

The proposed AM-MDP is a hybrid model developed by integrating MDP and MARL for threat mitigation. An MDP is defined by a tuple $\langle S, A, P, R, \gamma \rangle$, where $S \subseteq \mathbb{R}^n$ indicates a set of states representing the system, $A \subseteq \mathbb{R}^m$ refers to a set of actions available to the agent, $P(s'|s,a)$ denotes transition probability from state s to state s' after taking action a, $R(s,a) \in \mathbb{R}$ addresses the reward function for taking action a in state s, and $\gamma \in [0,1]$ states discount factor $(0 \le \gamma \le 1)$ to balance immediate and future rewards. The objective of MDP is to find a policy $\pi(a|s)$ that maximizes the expected cumulative reward as shown in Eq. (46).

$$V^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] \tag{46}$$

In MARL, multiple agents work collaboratively or competitively, and the state-action dynamics become multi-dimensional. For each agent i, S_i refers to the local state space for the agent i, A_i addresses the action space for the agent i, $R_i(s,a)$ signifies reward for the agent i, where $a = (a_1, a_2, \ldots, a_n)$ is the joint action of all agents, and $P(s'|s,a)$ points to the transition probability considering joint actions. The goal is to maximize each agent's value function $V_i^{\pi}(s)$ as defined in Eq. (47).The state s is enhanced to include contextual information C (threat severity, environment dynamics) as represented in Eq. (48).

$$V_i^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R_i(s_t, a_t)\right] \tag{47}$$

$$S' = S \times C \tag{48}$$

The reward function is adjusted to prioritize threat mitigation while penalizing delays as signified in Eq. (50), where λ specifies weight for successful mitigation, μ states the weight for penalties (delay or misaction), $C_{\text{mitigation}}$ stands for contextual reward for mitigating threats, and $C_{penalty}$ contextual penalty for failures.

$$R'(s,a) = R(s,a) + \lambda \cdot C_{\text{mitigation}} - \mu \cdot C_{penalty} \tag{50}$$

A centralized policy $\pi^*(a|s)$ is approximated by combining individual agents' policies using value decomposition as given in Eq.

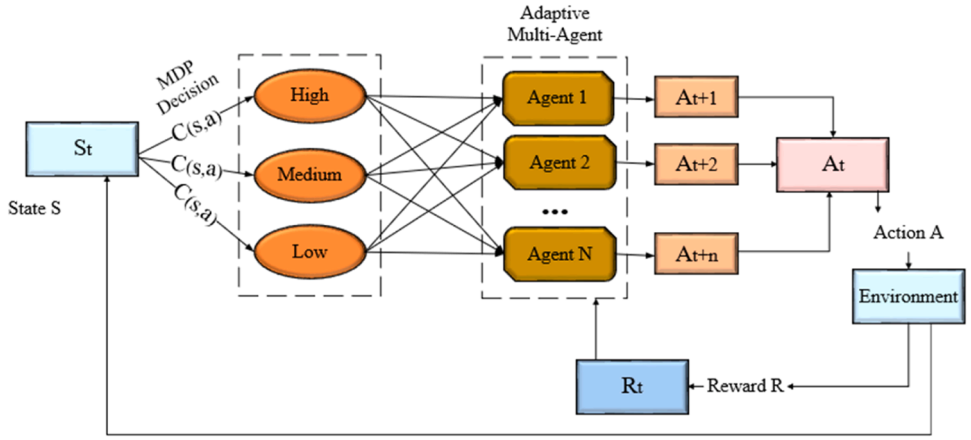


Fig. 5. Architecture of proposed AM-MDP.

(51), where n indicates agent count.

$$\pi^*(a|s) = \underset{\pi}{argmax} \sum_{i=1}^{n} V_i^{\pi}(s) \tag{51}$$

Now, the agents collaboratively decide on threat mitigation actions by dynamically balancing local and global objectives. The action for the agent i is shown in Eq. (52), in which $Q_i(s,a)$ indicates local action-value function, $Q_{global}(s,a)$ stands for global action-value function, and η defines the trade-off parameter.

$$a_i^* = \underset{a_i}{argmax} Q_i(s,a) + \eta \cdot Q_{global}(s,a) \tag{52}$$

The AM-MDP operates iteratively to evaluate the current state s_t and context C_t, and then, apply the joint policy π^* to select actions a_t. The transition and reward update takes place using Eq. (53).

$$s_{t+1} \sim P(s'|s_t,a_t),\ R'(s_t,a_t) \tag{53}$$

Fig. 5 shows the architecture of the proposed AM-MDP.

Once the threats are mitigated, the data is safely sorely in the cloud via blockchain technology.

3.6. Data transmission via blockchain

This process involves secure storage and transfer of mitigated threat data to the cloud by using the decentralized and tamper-proof nature of blockchain. Threat-mitigated data D is hashed to ensure integrity as shown in Eq. (54), where H_D addresses the unique hash of the data $D \in \mathbb{R}^n$, each mitigated event is hashed using Keccak-256 and stored via smart contracts, enabling integrity verification without exposing raw data. The hash H_D is sent to the Ethereum blockchain via a transaction T as formulated in Eq. (55).

$$H_D = Keccak256(D) \tag{54}$$

$$T = Ethereum_T(H_D, sender_{address}, contract_{address}) \tag{55}$$

Here, the transaction T ensures immutability and tamper-proof logging. The blockchain records the transaction in a block B, which includes H_D, the sender's address, and a timestamp as defined in Eq. (56).

$$B = [H_D, sender_{address}, timestamp] \tag{56}$$

The stored hash H_D is used as a reference in the cloud to verify the integrity of the actual data D before uploading.

4. Simulation results

The proposed MetaStack-APTNet is implemented in Python on an Intel Core® i5 processor (2.6 GHz, 16 GB RAM, 64-bit OS). Its performance is evaluated using metrics such as Accuracy, FPR, FNR, and MCC. The model is compared against baseline algorithms (XGBoost, LR, CNN, LSTM, BNN) and recent state-of-the-art models including GEAR-Net, MA-DQN-SBSA, ResNet, and DQN—CNN. To validate scalability and generalization, multiple datasets are employed: dapt2020 (https://www.kaggle.com/datasets/sowmyamyneni/dapt2020), IoT Intrusion Detection (https://www.kaggle.com/datasets/subhajournal/iotintrusion), KDDCup99 (https://www.kaggle.com/datasets/venkatakanumuru/kddcup99csv), and CIC IoT 2023 (https://www.kaggle.com/datasets/akashdogra/cic-iot-2023). This diverse dataset selection ensures comprehensive evaluation of MetaStack-APTNet across heterogeneous CPS environments, covering both legacy and contemporary attack scenarios.

4.1. Algorithmic analysis

The proposed MetaStack-APTNet is tested in comparison to the baseline models (XGBoost, LR, CNN, LSTM, BNN) and the available methods (GEAR-Net, MA-DQN-SBSA, ResNet, DQN—CNN) using 70:30 and 80:20 data splits. In all measures, accuracy, precision, F1-score, specificity, sensitivity, NPV, MCC, FPR and FNR, MetaStack-APTNet are always better than other models. In the 70:30 split, it reached a maximum accuracy of 97.67 and lowest false positive and false negative rates whereas the 80:20 split further enhanced the accuracy of 98.92 and lowest FPR. The findings show that MetaStack-APTNet is stronger and efficient in identifying and eliminating

Table 3
Performance of proposed MetaStack-APTNet for 70:30 learning samples with baseline methods.

Models	Accuracy	Precision	F1-Score	Specificity	Sensitivity	NPV	MCC	FPR	FNR
XGBoost	0.9589	0.952	0.9591	0.9513	0.9664	0.9659	0.9178	0.0487	0.0336
LR	0.9583	0.9468	0.9585	0.9463	0.9705	0.9703	0.9169	0.0537	0.0295
CNN	0.9616	0.949	0.9618	0.9484	0.9749	0.9746	0.9235	0.0516	0.0251
LSTM	0.954	0.9448	0.9544	0.9437	0.9642	0.9635	0.9082	0.0563	0.0358
BNN	0.9567	0.947	0.9571	0.9459	0.9675	0.9668	0.9136	0.0541	0.0325
Proposed MetaStack-APTNet	0.9767	0.9772	0.9767	0.9773	0.9762	0.9762	0.9534	0.0227	0.0238

APTs than the conventional baseline and advanced existing models. Tables 3 and 4 displays the Performance of proposed MetaStack-APTNet Learning Samples with Baseline Methods for 70:30 and 80:20 respectively.

Tables 5 and 6 represents the evaluation of the proposed model with other existing methods with a 70:30 and 80:20 data split, Whereas GEAR-Net and MA-DQN-SBSA perform competitively with high accuracy and precision, the proposed model significantly surpasses them in overall metrics.

Tables 7 and 8 illustrates the proposed MetaStack-APTNet models with Different Datasets of both data splits. In both 70:30 and 80:20 train-test splits, Dataset 1 always performed the best with the highest accuracy of up to 98.92, high precision, and low FPR whereas Dataset 2 recorded the lowest. On the whole, the model shows strong results on all datasets, and the 80:20 split results are better than 70:30.

Table 9 shows the resilience of the proposed MetaStack-APTNet model in identifying and mitigating various attacks measured through its Success Rate, Time to Detect, and Time to Mitigate metrics. Notably, the Success Rate evaluates the model's accuracy in correctly identifying specific attacks like DoS, APT, MITM, Ransomware, and others by achieving values often exceeding 95 %. Time to Detect quantifies the promptness of attack detection, with typical detection times ranging between 200 and 350 milliseconds, while Time to Mitigate measures the system's response effectiveness, generally within 400–700 milliseconds.

4.2. Explainability via SHAP

In the interest of interpretability, SHAP (SHapley Additive Explanations) is added to the MetaStack-APTNet pipeline. Each feature in the input data is assigned SHAP values to determine its impact on the end result of the classification. Findings showed that the frequency of PLC commands, authentication anomalies, and data rate bursts were always among the best predictive indicators of APTs. SHAP provide practical information to human analysts, allowing them to trust and be transparent in important industrial decision-making.

4.3. APT attack pathway case studies

Six typical attacks, namely DoS, APT, MITM, ransomware, phishing, and botnet, were studied in the form of case studies. In each of them, we examined detection confidence, mitigation latency, and SHAP-based contributing factors. In the case of APT, the model was able to detect multistage patterns of lateral movement and file integrity violations. The detection took place in 300 ms and the mitigation was completed in 600 ms. These realistic examples confirm the effectiveness and responsiveness of the system in real conditions of CPS.

4.4. Computational resource analysis

In order to determine deployability, resource consumption is measured during training and inference. MetaStack-APTNet used an average of 5.2 GB of GPU memory and 2.1 GB of system RAM during training on a system with Intel i7 CPU, 16 GB RAM and NVIDIA RTX GPU, compared to 3.4 GB CNN and 4.1 GB LSTM. The average time per epoch of training are 18.7 s and the inference latency per sample is approximately 200 ms, which is in line with real-time specifications. Computational cost is more expensive than single-model baselines, but the overhead is compensated by the high-performance improvements and the mitigation pipeline.

5. Discussion

The simulated environment is a model of a typical industrial Cyber-Physical System (CPS), such that a smart water treatment system, in which the common cyber threats, e.g., false data injection and DoS attacks, are injected. In this simulation, the Fuzzy Threat Scoring system is a dynamic system of measuring the threat level on the basis of anomalous sensor readings, communication anomalies, and operational context. The AM-MDP model is dynamic and changes with the threat environment and the system behavior, making the necessary adjustments to mitigation strategies in real time. The framework demonstrates how the threat scores change as the simulation unfolds, and how the AM-MDP learns optimal policies with respect to different attack scenarios. Quantitative measures like response latency, policy adjustment time, and stabilization of the threat level have been examined and presented to prove the viability and responsiveness of the system.

Table 4
Performance of proposed MetaStack-APTNet for 80:20 learning samples with baseline methods.

Models	Accuracy	Precision	F1-Score	Specificity	Sensitivity	NPV	MCC	FPR	FNR
XGBoost	0.9664	0.9533	0.9667	0.9527	0.9804	0.9801	0.9332	0.0473	0.0196
LR	0.9653	0.9553	0.9656	0.9547	0.9761	0.9758	0.9309	0.0453	0.0239
CNN	0.9686	0.9594	0.9687	0.9591	0.9782	0.9781	0.9374	0.0409	0.0218
LSTM	0.9643	0.957	0.9643	0.957	0.9716	0.9716	0.9286	0.043	0.0284
BNN	0.9621	0.9569	0.962	0.957	0.9673	0.9674	0.9243	0.043	0.0327
Proposed MetaStack-APTNet	0.9892	0.9912	0.989	0.9914	0.9869	0.9872	0.9783	0.0086	0.0131

Table 5
Performance of proposed MetaStack-APTNet for 70:30 learning samples with existing methods.

Models	Accuracy	Precision	F1-Score	Specificity	Sensitivity	NPV	MCC	FPR	FNR
GEAR-Net	0.9637	0.9498	0.9585	0.9498	0.9674	0.9674	0.9172	0.0502	0.0326
MA-DQN-SBSA	0.9675	0.957	0.9622	0.9573	0.9674	0.9676	0.9247	0.0427	0.0326
ResNet	0.9664	0.957	0.9611	0.9572	0.9653	0.9655	0.9225	0.0428	0.0347
DQN—CNN	0.9621	0.9673	0.962	0.9674	0.9569	0.957	0.9243	0.0326	0.0431
Proposed MetaStack-APTNet	0.9767	0.9772	0.9767	0.9773	0.9762	0.9762	0.9534	0.0227	0.0238

Table 6
Performance of proposed MetaStack-APTNet for 80:20 learning samples with existing methods.

Models	Accuracy	Precision	F1-Score	Specificity	Sensitivity	NPV	MCC	FPR	FNR
GEAR-Net	0.9702	0.9622	0.9648	0.9626	0.9675	0.9677	0.93	0.0374	0.0325
MA-DQN-SBSA	0.9783	0.989	0.9782	0.9891	0.9676	0.9681	0.9569	0.0109	0.0324
ResNet	0.9729	0.9676	0.9676	0.9677	0.9676	0.9677	0.9354	0.0323	0.0324
DQN—CNN	0.9783	0.989	0.9782	0.9891	0.9676	0.9681	0.9569	0.0109	0.0324

Table 7
Performance of proposed MetaStack-APTNet for 70:30 learning samples with different datasets.

Models	Accuracy	Precision	F1-score	Specificity	Sensitivity	NPV	MCC	FPR	FNR
Dataset 1	0.9767	0.9772	0.9767	0.9773	0.9762	0.9762	0.9534	0.0227	0.0238
Dataset 2	0.9567	0.9573	0.9573	0.956	0.9573	0.956	0.9134	0.044	0.0427
Dataset 3	0.9643	0.9609	0.965	0.9593	0.9691	0.9678	0.9285	0.0407	0.0309
Dataset 4	0.9632	0.9588	0.9639	0.9571	0.9691	0.9678	0.9264	0.0429	0.0309

Table 8
Performance of proposed MetaStack-APTNet for 80:20 learning samples with different datasets.

Datasets	Accuracy	Precision	F1-score	Specificity	Sensitivity	NPV	MCC	FPR	FNR
Dataset 1	0.9892	0.9912	0.989	0.9914	0.9869	0.9872	0.9783	0.0086	0.0131
Dataset 2	0.9691	0.971	0.9695	0.9704	0.9679	0.9672	0.9383	0.0296	0.0321
Dataset 3	0.9653	0.963	0.9661	0.9614	0.9692	0.9678	0.9307	0.0386	0.0308
Dataset 4	0.9686	0.9609	0.9691	0.9596	0.9774	0.9767	0.9373	0.0404	0.0226

Table 9
Performance of proposed MetaStack-APTNet in identifying and mitigating attacks to measure the resilience ability.

Attacks	Success Rate	Time to Detect	Time to Mitigate
DoS	98.5	250	500
APT	96.8	300	600
MITM	97.2	270	550
Ransomware	95.6	350	700
Phishing	99.1	200	400
Botnet	97.8	280	520

6. Conclusion

This paper introduced MetaStack-APTNet, a novel explainable, and edge-capable framework for APT detection and mitigation technique for CPS that combined an ensemble DL approach with advanced threat mitigation techniques. First, the massive network data and PLC logs were pre-processed using several preprocessing techniques. PCA was then utilized to reduce dimensionality while preserving significant patterns. To detect threats, the proposed MetaStack-APTNet model combined a stacked ensemble model (CNN with LSTM and BNN) and a hybrid meta-learner (XGBoost with LR). Additionally, the proposed DAFO was used to adjust the hyperparameters to increase the competence of the proposed MetaStack-APTNet model. The theoretical basis of DAFO was formalized, the ensemble architecture redesigned with adaptive fusion, and evaluation extended to real-world industrial datasets. Once the threats were identified, they were mitigated using the proposed AM-MDP model. Finally, blockchain technology was used to safely store the data in the cloud after the threats had been eliminated. The proposed MetaStack-APTNet exposed considerable computational time due to the integration of multiple DL models and hyperparameter tuning with DAFO. The inclusion of SHAP explainability and edge latency testing demonstrates both interpretability and deployability. Future work could focus on exploring lightweight ensemble architectures

Algorithm 1
Pseudocode of Developed DAFO.

Input:

Objective function $F(X)$, population size N, number of dimensions m, lower bound lb, upper bound ub, maximum iterations Max_{Iter}

Output:

Best solution X_{best} and the corresponding objective value F_{best}

Initialization:

Generate initial population X using Eq. (30)
Evaluate each x_i using the objective function $f(x_i)$
Set $x^* = \text{argmin} f(x_i)$
For each iteration $i = 1$ *to* N
  For each $d = 1$ *to* m
    Apply Eq. (30)
  End for
End for
Evaluate initial population
For each $i = 1$ *to* N
  $F_i = F(X_i)$
End for
Identify the initial best solution.
$X_{best} = X_{argmin}(F)$, $F_{best} = \min(F)$
For each $t = 1$ *to* Max_{Iter}
  Find better candidates in the population.
    Randomly select a region to forage
    Update position using exploration and exploitation formulas
  For each $i = 1$ *to* N
    Identify better candidates using Eq. (32)
    Randomly select a region to forage by selecting X_k
    from CA_i
    Update position using Eq. (33)
    Evaluate new position
    $F_i^{p_1} = F^{p_1}(X_i)$
    Replace $X_i = X_i^{p_1}$ if $F_i^{p_1} \leq F_i$
  End for
End for
Flooding and Removal Stage:
For each $i = 1$ *to* N
  Apply the Flooding operator to introduce randomness using Eq. (36)
  Apply the Removal operator to push unfit individuals toward the worst solution using Eq. (37)
  Update position using Eq. (38)
  Evaluate new position
  $F_i^{p_2} = F^{p_2}(X_i)$
  Replace $X_i = X_i^{p_2}$ if $F_i^{p_2} \leq F_i$
End for
Update the global best solution
If $min(F) < F_{best}$
$X_{best} = X_{argmin}(F)$, $F_{best} = min(F)$
Return Best Solution
Return X_{best} and F_{best}

for real-time APT detection in resource-constrained environments and expand the SHAP toolkit for more granular forensic auditing. Besides, incorporate explainable AI techniques to improve the interpretability of MetaStack-APTNet for actionable insights. Beyond CPS, the MetaStack-APTNet framework has potential applications in smart city infrastructure, healthcare IoT systems, and financial systems, where robust and adaptive APT detection is critical.

Declaration of competing interest

The authors declare that we have no conflict of interest.

Data availability

No data was used for the research described in the article.

References

[1] Awotunde JB, Gaber T, Prasad LN, Folorunso SO, Lalitha VL. Privacy and security enhancement of smart cities using hybrid deep learning-enabled blockchain. Scalable Comput: Pract Exp 2023;24(3):561–84.
[2] bin Zainuddin AA, Sairin H, Mazlan IA, Muslim NN, Sabarudin WA. Enhancing IoT security: a synergy of machine learning, artificial intelligence, and blockchain. Data Sci Insights 2024;2(1).
[3] Odeh A, Abu Taleb A. Ensemble-based deep learning models for enhancing IoT intrusion detection. Appl Sci 2023;13(21):11985.
[4] Kavitha D, Thejas S. Ai enabled threat detection: leveraging artificial intelligence for advanced security and cyber threat mitigation. IEEE Access 2024.
[5] Tooki OO, Popoola OM. A critical review on intelligent-based techniques for detection and mitigation of cyberthreats and cascaded failures in cyber-physical power systems. Renew Energy Focus 2024;51:100628.
[6] Manias DM, Saber AM, Radaideh MI, Gaber AT, Maniatakos M, Zeineldin H, El-Saadany EF. Trends in smart grid cyber-physical security: components, threats and solutions. IEEE Access 2024.
[7] Nour AA, Mehbodniya A, Webber JL, Bostani A, Shah B, Ergashevich BZ. Optimizing intrusion detection in industrial cyber-physical systems through transfer learning approaches. Comput Electr Eng 2023;111:108929.
[8] Wang Z. Cyber digital twin with deep learning model for enterprise products management. Wirel Pers Commun 2024:1–30.
[9] Waheed N, He X, Ikram M, Usman M, Hashmi SS, Usman M. Security and privacy in IoT using machine learning and blockchain: threats and countermeasures. ACM comput surv (csur) 2020;53(6):1–37.
[10] Alam K, Al Imran M, Mahmud U, Al Fathah A. Cyber attacks detection and mitigation using machine learning in smart grid systems. J Sci Eng Res 2024;12.
[11] Arefin S, Chowdhury M, Parvez R, Ahmed T, Abrar AS, Sumaiya F. Understanding APT detection using machine learning algorithms: is superior accuracy a thing? In2024 IEEE Int Conf Electro Inf Technol (eIT) 2024:532–7.
[12] Neuschmied H, Winter M, Stojanović B, Hofer-Schmitz K, Božić J, Kleb U. Apt-attack detection based on multi-stage autoencoders. Appl Sci 2022;12(13):6816.
[13] Fahad M, Airf H, Kumar A, Hussain HK. Securing against apts: advancements in detection and mitigation. BIN: Bull Inform 2023;1(2).
[14] Lee K, Lee J, Yim K. Classification and analysis of malicious code detection techniques based on the APT attack. Appl Sci 2023;13(5):2894.
[15] Zhang K, Shi Y, Karnouskos S, Sauter T, Fang H, Colombo AW. Advancements in industrial cyber-physical systems: an overview and perspectives. IEEE Trans Ind Inform 2022;19(1):716–29.
[16] Lee J, Kundu P. Integrated cyber-physical systems and industrial metaverse for remote manufacturing. Manuf Lett 2022;34:12–5.
[17] Rawash YZ, Al-Naami B, Alfraihat A, Owida HA. Advanced low-pass filters for signal processing: a comparative study on Gaussian, Mittag-Leffler, and Savitzky-Golay filters. Math Model Eng Probl 2024;11(7).
[18] Pukkala T, Aquilué N, Just A, Corbera J, Trasobares A. Developing kNN forest data imputation for Catalonia. J For Res 2024;35(1):80.
[19] Geem D, Hercules D, Pelia RS, Venkateswaran S, Griffiths A, Noe JD, Dotson JL, Snapper S, Rabizadeh S, Rosh JR, Baldassano RN. Progression of pediatric Crohn's disease is associated with anti–tumor necrosis factor timing and body mass index Z-score normalization. Clin Gastroenterol Hepatol 2024;22(2):368–76.
[20] Hammoumi D, HS Al-Aizari, Alaraidh IA, Okla MK, Assal ME, Al-Aizari AR, Moshab MS, Chakiri S, Bejjaji Z. Seasonal variations and assessment of surface water quality using water quality index (WQI) and principal component analysis (PCA): a case study. Sustainability 2024;16(13):5644.
[21] Alenazi M, Mishra S. Cyberatttack detection and classification in IIoT systems using XGBoost and Gaussian Naïve Bayes: a comparative study. Engineering. Technol Appl Sci Res 2024;14(4):15074–82.
[22] Kolukisa B, Dedeturk BK, Hacilar H, Gungor VC. An efficient network intrusion detection approach based on logistic regression model and parallel artificial bee colony algorithm. Comput Stand Interfaces 2024;89:103808.
[23] Najar AA, Naik SM. Cyber-secure SDN: a CNN-based approach for efficient detection and mitigation of DDoS attacks. Comput Secur 2024;139:103716.
[24] Liu Y, Xu L, Yang S, Zhao D, Li X. Adversarial sample attacks and defenses based on LSTM-ED in industrial control systems. Comput Secur 2024;140:103750.
[25] Gill KS, Saxena S, Sharma A, Dhillon A. GTBNN: game-theoretic and bayesian neural networks to tackle security attacks in intelligent transportation systems. Clust Comput 2024;27(8):11645–65.